

Kártyafelismerő

Projekt-dokumentáció

Tartalom

Bevezetés	1
Elméleti háttér	2
Mélytanulás.....	2
Neurális hálók alapjai	2
Rétegek	2
Deep Neural Networks (DNN)	3
Konvolúciós neurális hálók (CNN)	3
CNN felépítése.....	4
CNN működése	4
Objektumdetektálás	8
Szintetikus adatkészletek	10
A megvalósítás terve és kivitelezése	11
A generate_cards_dataset.py program bemutatása	11
Training.py program bemutatása	19
Eredmények	21
Metrikák magyarázata	25
A modell használata predikcióra	28
Hivatkozások	29

Bevezetés

A Gépi látás tárgy keretében egy olyan rendszert készítek, amely képeken vagy videókon automatikusan felismeri és detektálja a francia kártyapakli lapjait. A projekt célja egy teljes gépi látás pipeline megvalósítása, amely szintetikus adatkészlet generálásával kezdődik, majd egy mély neurális háló tanításával folytatódik, végül pedig a tanított modell segítségével képeken és videókon történő objektumdetektálással zárul.

Projekt pipeline:

1. Dataset generálás

1. Asset betöltés
 - Kártyák
 - Háttérképek
2. Osztályrendszer definiálása
3. Random háttér kiválasztása
4. Random kártyák kiválasztása
5. Augmentáció
 - skálázás
 - forgatás
6. Elhelyezési szabályok
 - Random pozíció, de
 - ne essen ki a képből
 - ne legyen teljes takarás
7. Alpha blending
8. Bounding box számítás
9. YOLO label mentés
10. Train/Val/Test split
 - 80% Train
 - 10% Val
 - 10% Test

2. Gyors vizuális ellenőrzés

- 10-20 random kép bbox kirajzolás és class ellenőrzés

3. 50 epoch training

4. Eredmények dokumentálása

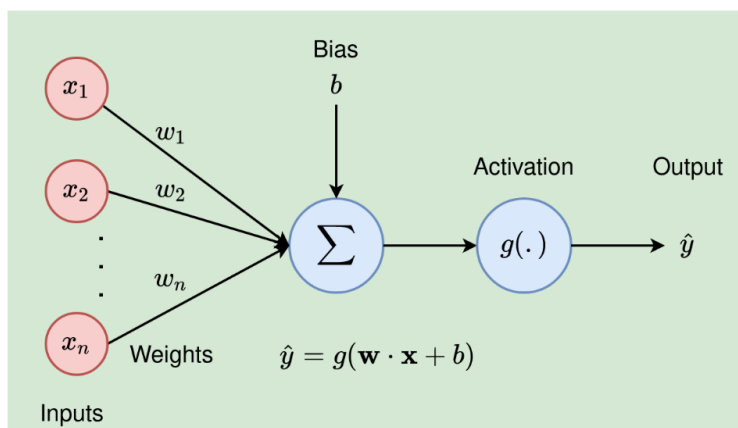
Elméleti háttér

Mélytanulás

Neurális hálók alapjai

A neurális hálózatok olyan mesterséges modellek, amelyek célja az adatok közötti összefüggések felismerése. Alapvető egységük a neuron, amely az emberi idegsejt mintájára készült. Egy neuron bemeneteket fogad ($x_1, x_2, \dots, x_n$), ezeket súlyokkal szorozza ($w_1, w_2, \dots, w_n$) amely folyamat skaláris szorzat formájában történik, majd egy aktivációs függvény segítségével határozza meg a kimenetét. [1]

Matematikailag egy neuron működése az alábbi ábrával írható le:

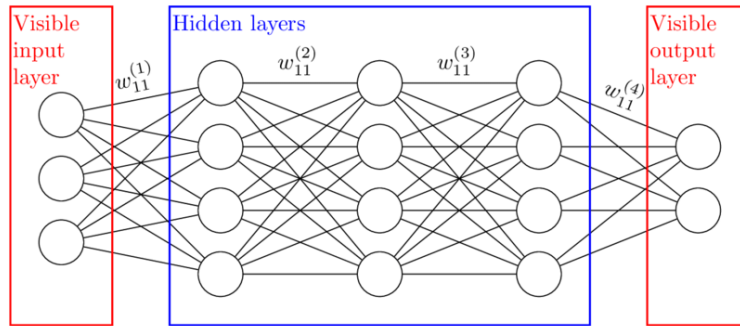


Neuron működése [Exploring the Power of Neural Networks | by Vibhura | Medium](#) 2026.03.07.

Rétegek

Egy neurális hálózat több rétegből áll, ahol minden réteg tetszőleges számú neuronnal, és a neurális hálózat pedig tetszőleges számú réteggel rendelkezhet. Egy réteg minden neuronja kapcsolatban áll a szomszédos rétegek minden neuronjával. Az információt a rétegeken keresztül dolgozza fel, ahol az egyes rétegek bemenete az előző rétegek kimenete.

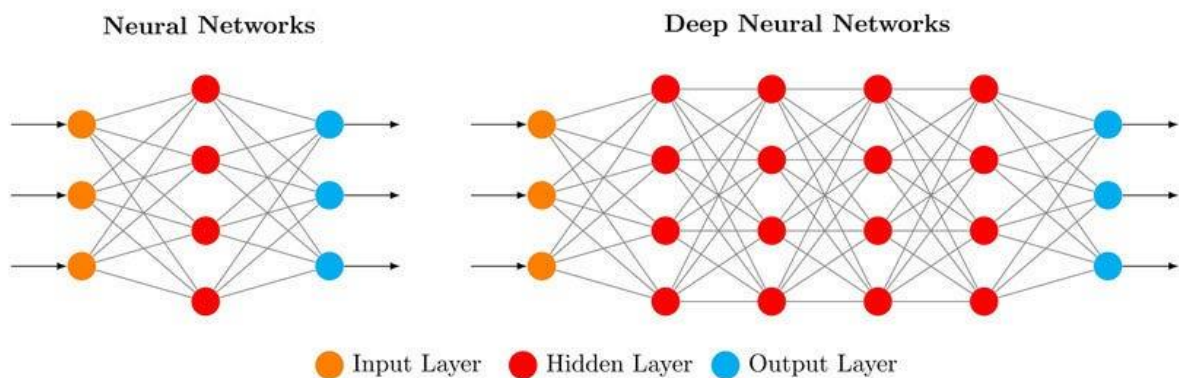
- **Bemeneti réteg** fogadja a nyers adatokat, és minden neuronja a bemeneti adat egy jellemzőjét képviseli, például egy kép egy pixeljének számmá alakított értékét.
- **Rejtett rétegek**ben történik a tanulás a súlyok, bias, és aktivációs függvény segítségével.
- **Kimeneti réteg** szolgáltatja az eredményt. A neuronok száma a lehetséges kimeneti osztályok számától függ. Bináris osztályozás esetén általában 1 neuron van, sigmoid aktivációval, többosztályos esetben pedig annyi neuron, ahány osztály, softmax aktivációval. [2]



Neurális hálózat rétegei [An example of a deep learning neural network with 3 hidden layers. For... | Download Scientific Diagram](#) 2026.03.07.

Deep Neural Networks (DNN)

A DNN egy olyan neurális hálózat amelyben a bemeneti, és a kimeneti réteg között egynél több rejtett réteg található. Elvileg egy 1 rejtett réteggel rendelkező hálózat is képes bármilyen folytonos függvényt közelíteni, de ha a tanulandó kapcsolat nagyon komplex, akkor erre csak úgy lenne képes, ha rengeteg neurona lenne, ami nagyon nagy hálót, nehezen tanítható rendszert eredményezne. DNN esetében az első rétegek egyszerűbb mintákat (pl. élek, szögek a képfeldolgozásban), a középsők bonyolultabb kombinációkat, a felsőbb rétegek pedig nagyon komplex reprezentációkat tanulnak meg. Egyszerű példával, az 1 rétegű háló olyan lenne, mintha egyetlen nagyon bonyolult szabályt akarnánk írni, míg a DNN esetén sok kisebb, egymásra épülő szabályt. [2]



Mély neurális hálózat [Deep Learning. Surf the Neural Waves! | by Surya R | Medium](#) 2026.03.07.

Konvolúciós neurális hálók (CNN)

A DNN-ek, és a CNN-ek mind mesterséges neurális hálók, amelyek képesek komplex mintázatok felismerésére. A köztük lévő alapvető különbség a bemeneti adatok feldolgozásának módjában rejlik. A DNN-ekben minden neuron az előző réteg összes neuronjához teljesen kapcsolódik (fully connected layer), tehát például egy 28x28 pixeles kép 784 bemenetet jelent a háló számára, és minden képpontot külön egységként kezel, nem pedig összefüggő mintázatként. Ezzel szemben a CNN-ek kifejezetten képek, és más rács struktúrájú adatok feldolgozására lettek kifejlesztve.

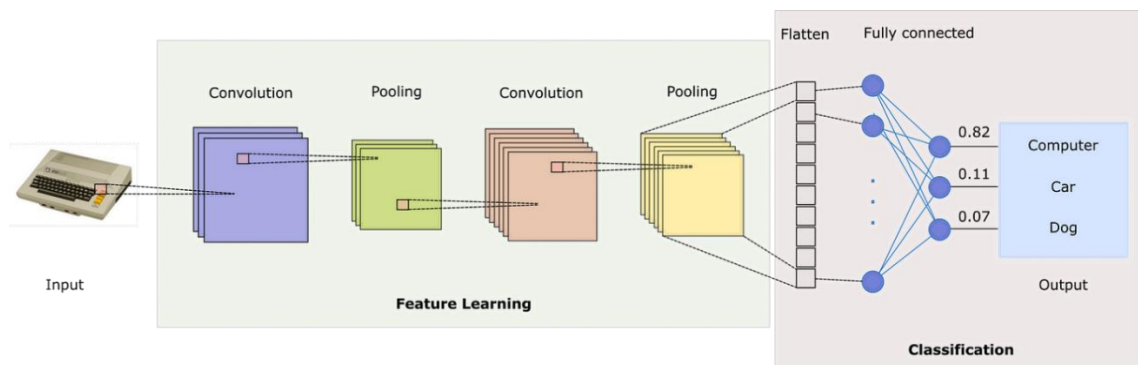
A CNN megőrzi a bemeneti kép térbeli struktúráját, és képes lokális minták, például élek, textúrák vagy formák felismerésére.

CNN felépítése

A CNN tipikus felépítése a következő rétegekből áll:

- **Konvolúciós rétegek:** A bemeneti adatokból jellemzőket (feature-öket) emelnek ki különböző tanulható szűrők segítségével.
- **Pooling rétegek:** A jellemzőtérképek méretét csökkentik, miközben a legfontosabb információkat megtartják.
- **Flatten réteg:** A többdimenziós jellemzőtérképet egydimenziós vektorra alakítja, hogy az a teljesen összekapcsolt rétegek számára feldolgozható legyen.
- **Fully connected réteg:** A korábban kinyert jellemzők alapján végső döntéseket hoznak.

A konvolúciós, és pooling rétegekből álló feature learning szakasz megtanulja a bemeneti adatok jellemzőit(pl. élek, textúrák, alakzatok) melyeket a flatten, és fully connected rétegekből álló classification szakasz a döntéshozatalhoz használ fel. [3]

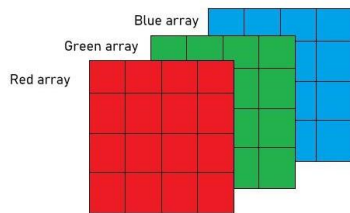


CNN felépítése [What is Convolutional Neural Network — CNN \(Deep Learning\) | Microsoft Community Hub](#)
2026.03.07.

CNN működése

Input

A CNN-ek működésének megértéséhez először érdemes átgondolni, hogyan tekinti a számítógép a képeket. Egy kép a számítógép számára számok mátrixa, minden egyes pixelhez numerikus értékek tartoznak, amelyek a szín és fényerő információkat kódolják. Színes (RGB) képeknél minden pixelhez 3 szám tartozik, a vörös, zöld, és kék csatornához, a szürkeárnyaltos képeknél pedig minden pixelhez egyetlen érték tartozik amely a fény intenzitását jelöli az adott ponton. A CNN bemeneti rétege a képet egy háromdimenziós tömbként fogadja. Ennek mérete: magasság × szélesség × csatornák száma. RGB képeknél a csatornák száma 3 (vörös, zöld, kék), így például egy 28×28 pixeles RGB kép bemeneti formája 28×28×3, míg szürkeárnyaltos képeknél, ahol minden pixelhez csak egyetlen fényerő érték tartozik, a bemenet 28×28×1 lesz. [4]

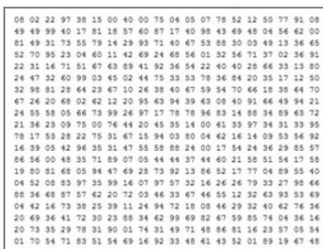


RGB csatornák [Convolutional Neural Network\(CNN\) in deep learning : | by NEELLAPALLI PALLAVI | Medium](#) 2026.03.07.

What I see



What a computer sees

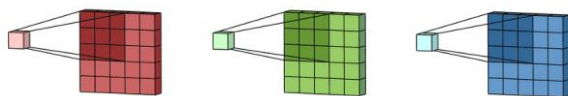


What a computer sees [PowerPoint-presentatie](#) 2026.03.07.

Konvolúciós rétegek

A CNN gyakorlatilag legfontosabb építőköve a konvolúciós réteg, melynek célja lokális jellemzők kinyerése a bemeneti képből. Ez a folyamat a kernelnek nevezett speciális szűrők segítségével valósul meg. A kernel egy kis méretű számmátrix (pl. 2x2 vagy 3x3) amely az aktuális helyzetében lévő képrészletet feldolgozza a következő módon:

- A kernel pixelrészletenként végighalad a képen
- Minden pozícióban a kernel és a képrészlet elemeit szorzás-összegzés művelettel kivonjuk, és átadjuk egy kimeneti mátrixnak amit jellemző térképnek(feature map) nevezünk.



Kernel mozgása [Convolutional Neural Networks In Depth | by Rajesh Katta | Medium](#) 2026.03.07.

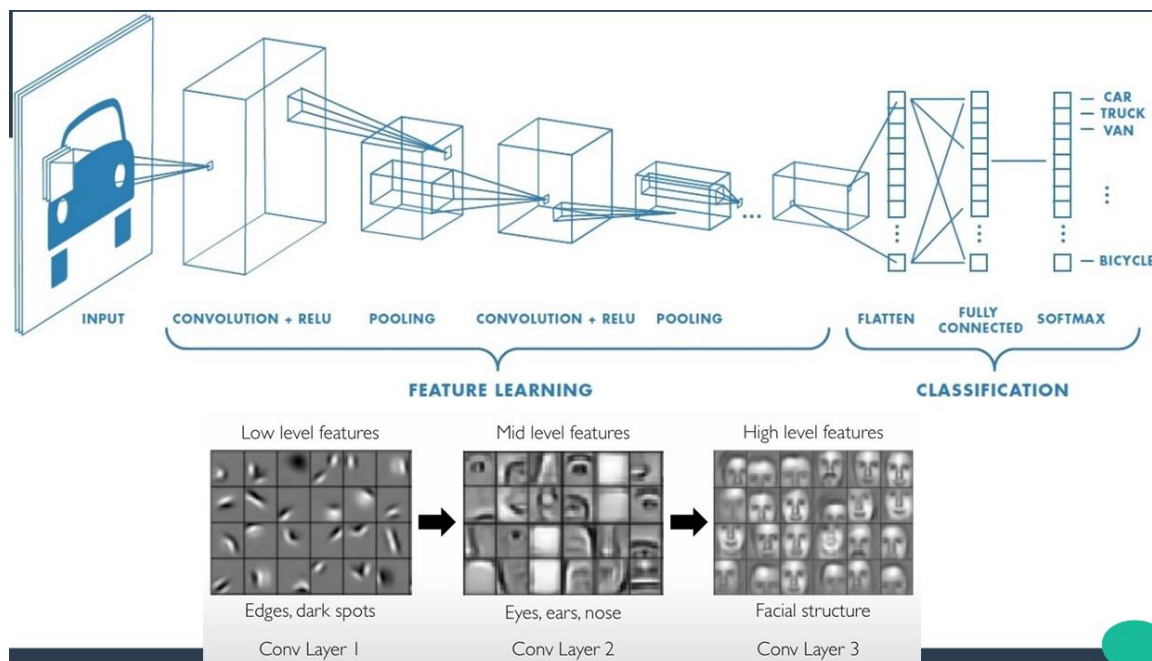
Input		Kernel		Output																	
<table border="1"> <tr><td>0</td><td>1</td><td>2</td></tr> <tr><td>3</td><td>4</td><td>5</td></tr> <tr><td>6</td><td>7</td><td>8</td></tr> </table>	0	1	2	3	4	5	6	7	8	*	<table border="1"> <tr><td>0</td><td>1</td></tr> <tr><td>2</td><td>3</td></tr> </table>	0	1	2	3	=	<table border="1"> <tr><td>19</td><td>25</td></tr> <tr><td>37</td><td>43</td></tr> </table>	19	25	37	43
0	1	2																			
3	4	5																			
6	7	8																			
0	1																				
2	3																				
19	25																				
37	43																				

Szorzás-összegzés művelet [7.2. Convolutions for Images — Dive into Deep Learning 1.0.3 documentation](#) 2026.03.07.

[5]

A CNN hierarchikus módon egyre komplexebb jellemzőket tanul a hálózat mélységének növekedésével.

- Első néhány réteg:
 - Alapvető vizuális mintázatokat tanulnak meg pl. vízszintes és függőleges élek, átlós vonalak, egyszerű textúrák, színátmenetek stb.
- Mélyebb rétegek:
 - A korábbi rétegek kimeneteire építve, a receptive field fokozatos növekedése (egyre nagyobb képrészleteket látnak egyszerre) miatt komplexebb, globálisabb jellemzőket tanulnak meg pl. formák (körív, téglalap stb.), specifikus objektumrészletek (orr, szem, fül az arcfelismerésben), egész objektumok (pl. autó, kutya, ember stb.). [6]

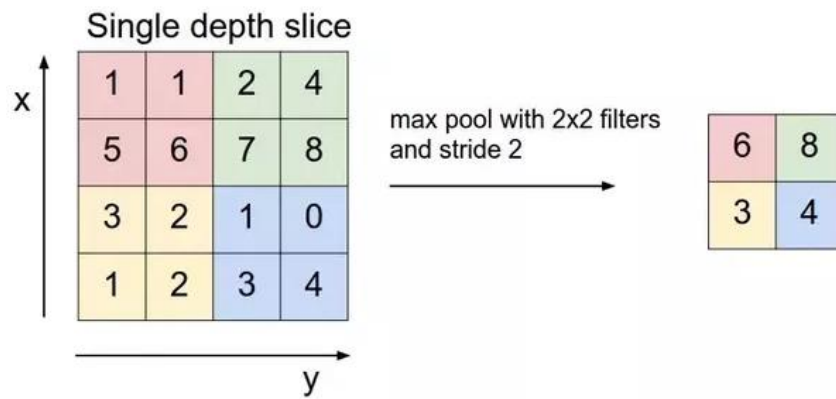


Feature learning [Convolut_network_Benyo.pdf](#) 2026.03.07.

Pooling réteg

A konvolúciós rétegeket gyakran pooling rétegek követik. A pooling célja a feature map méretének csökkentése úgy, hogy közben a legfontosabb információk megmaradjanak.

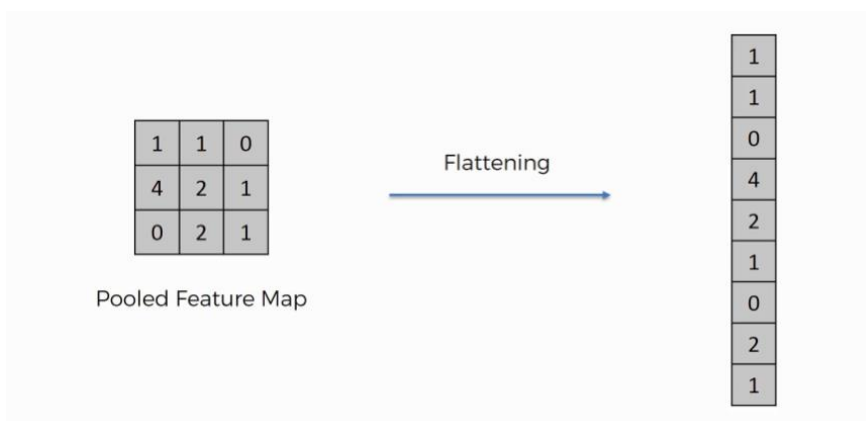
A leggyakoribb pooling művelet a **max pooling**, amely egy adott területen belül a legnagyobb értéket választja ki. Ez csökkenti a számítási igényt, valamint növeli a modell robusztusságát kisebb eltolásokkal vagy zajjal szemben. [7]



Max pooling [The max-pooling operation \[28\] | Download Scientific Diagram](#) 2026.03.07.

Flatten réteg

A konvolúciós, és pooling rétegek után az adat többdimenziós tömbként van jelen. Mivel a következő réteg egy fully connected layer, amely egydimenziós vektort vár bemenetként, a flattening réteg ezt az adatot egydimenziós vektorra alakítja. A flattening csak átalakítja az adatszerkezetet, az információ tartalma nem változik. [8]



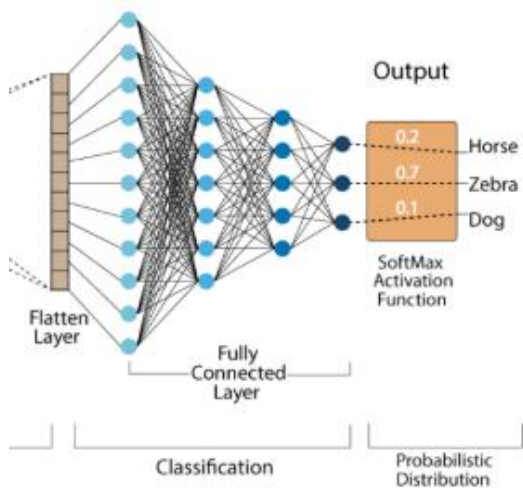
Flatten réteg [Flattening CNN layers for Neural Network and basic concepts | by Muhammad Shoaib Ali | Medium](#) 2026.03.07.

Fully connected réteg

A CNN utolsó rétege a Fully connected réteg. Ez a réteg a flatten réteg kimenetét dolgozza fel, ahol az összes bemeneti neuron kapcsolódik az összes kimeneti neuronhoz, és osztályozási döntést hoz. A kimeneti réteg neuronjainak száma a feladattól függ:

- bináris osztályozás esetén 1 neuron, és Sigmoid aktivációs függvény.

több osztályos osztályozás esetén(pl. 10 kategória) 10 neuron, és Softmax aktivációs függvény. A Softmax kimenete valószínűségi eloszlás, amely megmutatja, hogy az input kép melyik kategóriába tartozik a legnagyobb valószínűséggel. [3]



Fully Connected Layer [What is Convolutional Neural Network — CNN \(Deep Learning\) | Microsoft Community Hub](#) 2026.23.07.

Objektumdetektálás

Az objektumdetektálás lényegében a klasszifikáció kiterjesztése, amelyben a modellnek nemcsak azt kell eldöntenie, hogy milyen objektumok vannak a képen, hanem azt is, hogy azok hol helyezkednek el. A lokalizáció általában egy téglalap alakú jelöléssel, úgynevezett **bounding box** segítségével történik. [9]

Egy objektumdetektáló modell kimenete tipikusan a következő információkat tartalmazza:

- objektum osztálya
- bounding boxok
- a detektálás megbízhatóságát mutató confidence score [10]



Objektum detektáló kimenete (saját kép) 2026.03.07.

A bounding box koordinátáit általában a kép méretéhez viszonyított normalizált formában adják meg. YOLO formátumban a dobozt a következő paraméterek írják le:

- a bounding box középpontjának x koordinátája
- a bounding box középpontjának y koordinátája
- a bounding box szélessége
- a bounding box magassága [11]

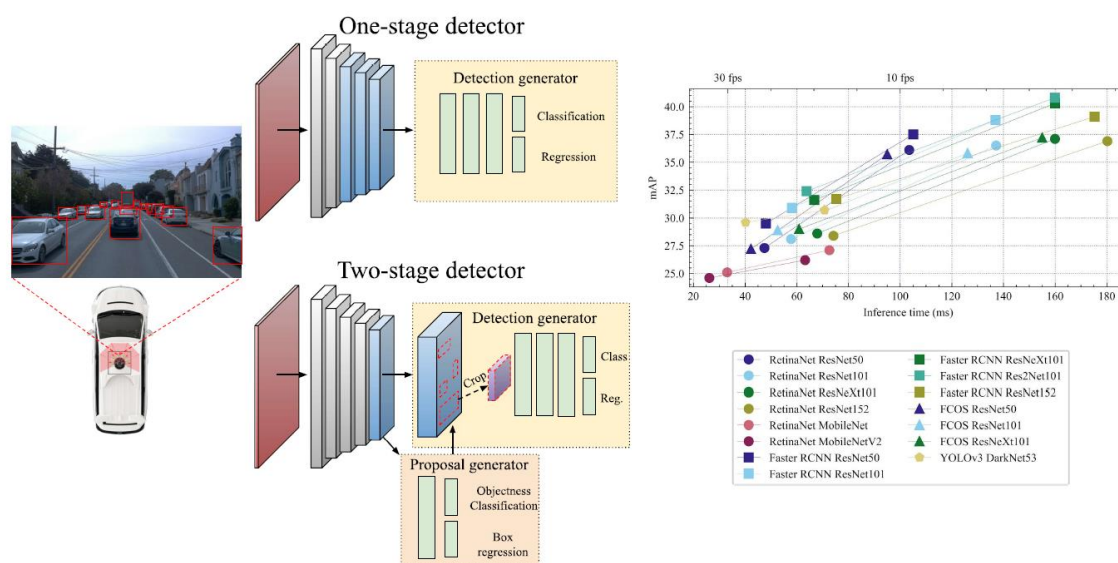


Bounding box paraméterek [Object Detection Datasets Overview - Ultralytics YOLO Docs](#) 2026.03.07.

Az objektumdetektáló módszerek két fő csoportra oszthatók.

Az úgynevezett **kétlépcsős (two-stage) módszerek**, például az R-CNN és a Faster R-CNN, először lehetséges objektum régiókat generálnak, majd ezekre külön osztályozást végeznek.

A **egylépcsős (single-stage) módszerek**, például a YOLO, egyetlen neurális háló segítségével közvetlenül prediktálják az objektumok helyét és osztályát. Ez a megközelítés jelentősen gyorsabb működést tesz lehetővé, ezért gyakran használják valós idejű alkalmazásokban. [12]



Egylépcsős és kétlépcsős módszerek [On the Performance of One-Stage and Two-Stage Object Detectors in Autonomous Vehicles Using Camera Data 2026. 03. 07.](#)

A jelen projektben a **YOLOv8n (nano)** modell került alkalmazásra. Ennek elsődleges oka, hogy a modell kifejezetten kis számítási igényű, gyorsan tanítható és kis méretű hálózat, amely alkalmas kisebb méretű vagy kísérleti adatkészleteken történő modellezésre, miközben nem megy a pontosság rovására. A YOLOv8n modell körülbelül 3 millió paraméterrel rendelkezik, ami jelentősen kisebb, mint a nagyobb YOLO modellek esetében. Ennek köszönhetően a modell:

- gyorsabban tanítható
- kevesebb GPU memóriát igényel
- kisebb számítási kapacitással is futtatható
- alkalmas valós idejű alkalmazásokra.

[13]

Szintetikus adatkészletek

A mélytanulási modellek hatékony tanításához általában nagy mennyiségű, megfelelően annotált adatra van szükség. Az objektumdetektálási feladatok esetében ez azt jelenti, hogy minden egyes képen meg kell jelölni az objektumok pozícióját és osztályát. Az ilyen adatkészletek létrehozása azonban jelentős idő- és erőforrásigénnyel jár, mivel az annotáció gyakran manuális munkát igényel.

A szintetikus adatok generálása során gyakran alkalmaznak különböző véletlenszerű transzformációkat annak érdekében, hogy növeljék az adatkészlet változatosságát, így a neurális háló többféle megjelenési formával találkozhat a tanítás során. Ezek közé tartozhat például:

- az objektumok véletlenszerű elhelyezése a képen,
- méretezés,
- forgatás,
- háttérképek változtatása,
- részleges takarás.

A jelen projektben a szintetikus adatkészlet úgy készült, hogy a kártyalapok PNG formátumú képei véletlenszerű háttérképekre kerültek elhelyezésre. A kártyák mérete, forgatása és pozíciója véletlenszerűen változott minden generált kép esetében, és a címkék is automatikusan generálódtak.

A szintetikus adatok egyik ismert korlátja, hogy a generált képek gyakran eltérnek a valós környezetben készült fényképektől, amely miatt a modell teljesítménye valós adatokon romolhat. [14]

A megvalósítás terve és kivitelezése

A generate_cards_dataset.py program bemutatása

A projekt első gyakorlati lépése a tanításhoz szükséges adathalmaz előállítása volt. Ezt a feladatot a generate_cards_dataset.py program valósítja meg. A program feladata, hogy a rendelkezésre álló kártya- és háttérképekből szintetikus, a tanításhoz, validációhoz és teszteléshez felhasználható képeket készítsen. A kimenet egy teljes train, val és test struktúrájú dataset.

Könyvtárak importálása

A program elején a szükséges könyvtárak importálása történik.

```
1 import cv2
2 from pathlib import Path
3 import random
4 import numpy as np
```

generate_cards_dataset.py részlet (saját kép) 2026.03.13.

A `cv2` könyvtár az OpenCV csomag, amelyet a képek beolvasására, átméretezésére, forgatására használok.

A `Path` a fájlrendszerben történő útvonalkezelést egyszerűsíti.

A `random` modult a véletlenszerű háttér-, kártya-, méret- és pozícióválasztásra használok.

A `numpy` a képmátrixok kezeléséhez szükséges.

Bemeneti útvonalak

A következő rész a bemeneti és kimeneti mappák meghatározását végzi.

```

6 CARDS_DIR = Path("assets/cards")
7 TEXTURE_DIR = Path("assets/texture")
8 DATASET_DIR = Path("datasets/yolo_cards")

```

generate_cards_dataset.py részlet (saját kép) 2026.03.13.

A `CARDS_DIR` mappa tartalmazza a kártyalapok PNG képeit.

A `TEXTURE_DIR` mappa a háttérként használt képeket tartalmazza.

A `DATASET_DIR` a generált YOLO dataset célmappája.

Ezután a program listázza az összes elérhető kártya- és háttérképet.

```

10 card_paths = list(CARDS_DIR.glob("*.png"))
11 bg_paths = list(TEXTURE_DIR.glob("*.jpg"))

```

generate_cards_dataset.py részlet (saját kép) 2026.03.13.

Osztályrendszer létrehozása

Mivel a modellnek minden egyes kártyalapot külön osztályként kell felismernie, szükség van egy osztályrendszer definiálására.

```

14 class_names = sorted([p.stem for p in card_paths])
15 class_to_id = {name: i for i, name in enumerate(class_names)}

```

generate_cards_dataset.py részlet (saját kép) 2026.03.13.

A `class_names` lista a kártyaképek fájlneveiből áll össze a kiterjesztés nélkül.

A `class_to_id` szótár minden kártyanévhez egy egész számot rendel. Ez az azonosító kerül majd az annotációs fájlok első oszlopába.

Cards.yaml fájl létrehozása

A YOLO tanításhoz szükség van egy konfigurációs fájlra, amely leírja a dataset felépítését. Ezt a program automatikusan hozza létre.

```

yaml_path = Path("cards.yaml")
with open(yaml_path, "w") as f:
    f.write("path: datasets/yolo_cards\n")
    f.write("train: images/train\n")
    f.write("val: images/val\n")
    f.write("test: images/test\n\n")
    f.write("names:\n")
    for i, name in enumerate(class_names):
        f.write(f" {i}: {name}\n")

```

generate_cards_dataset.py részlet (saját kép) 2026.03.13.

A `card.yaml` fájl tartalmazza:

- a dataset gyökérmappáját,
- a train, validation, test képek helyét,
- és az osztályok sorszám-név megfeleltetését.

Ez a fájl teszi lehetővé, hogy a YOLO helyesen értelmezze az elkészült adathalmazt.

Képek betöltése

A következő lépés a forrásképek memóriába töltése.

```
29 cards = {p.stem: cv2.imread(str(p), cv2.IMREAD_UNCHANGED) for p in card_paths}
30 backgrounds = [cv2.imread(str(p)) for p in bg_paths]
```

generate_cards_dataset.py részlet (saját kép) 2026.03.13.

A kártyák betöltése során az `IMREAD_UNCHANGED` használata fontos, mert így az OpenCV megtartja az alpha csatornát. Ez szükséges ahhoz, hogy a kártyák átlátszó háttérrel kerüljenek a háttérképre.

A program fix képméretet használ. Minden generált kép 640×640 pixeles lesz, így a dataset egységes méretű mintákból áll.

```
32 IMG_SIZE = 640
```

generate_cards_dataset.py részlet (saját kép) 2026.03.13.

Kártyák forgatása

A `rotate()` függvény feladata a kártyaképek elforgatása.

```
34 def rotate(card, angle):
35     h, w = card.shape[:2]
36     M = cv2.getRotationMatrix2D((w/2, h/2), angle, 1.0)
37
38     cos = abs(M[0, 0])
39     sin = abs(M[0, 1])
40
41     new_w = int(h * sin + w * cos)
42     new_h = int(h * cos + w * sin)
43
44     M[0, 2] += new_w / 2 - w / 2
45     M[1, 2] += new_h / 2 - h / 2
46
47     return cv2.warpAffine(card, M, (new_w, new_h), borderValue=(0, 0, 0, 0))
```

generate_cards_dataset.py – rotate függvény (saját kép) 2026.03.13.

A `h, w = card.shape[:2]` lekéri a kép magasságát és szélességét.

A `cv2.getRotationMatrix2D` létrehoz egy 2x3-as elforgatási mátrixot (M). A forgatás középpontja a kép közepe (w/2, h/2), a szög `angle`, a méretezés pedig 1.0 (eredeti méret).

where

$$\begin{bmatrix} \alpha & \beta & (1 - \alpha) \cdot \text{center.x} - \beta \cdot \text{center.y} \\ -\beta & \alpha & \beta \cdot \text{center.x} + (1 - \alpha) \cdot \text{center.y} \end{bmatrix}$$

$$\begin{aligned} \alpha &= \text{scale} \cdot \cos \text{angle}, \\ \beta &= \text{scale} \cdot \sin \text{angle} \end{aligned}$$

Elforgatási mátrix [OpenCV: Geometric Image Transformations](#) 2026.03.08.

[15]

A `new_w` és `new_h` megadják az új befoglaló téglalp méreteit, így nem maradnak le az eredeti kép sarkai a képről.

Mivel a kép mérete megnőtt, a forgatási középpont eltolódna. Az `M[0, 2]` és `M[1, 2]` értékek módosításával a képet az új vászon közepére pozicionálja.

`cv2.warpAffine` Elvégzi a tényleges transzformációt. Paraméterként fogadja a bemeneti képet (`card`), transzformációs mátrixot (M), kimeneti kép méretét (`new_w` és `new_h`), és a `borderValue=(0, 0, 0, 0)`-t ami a forgatás után keletkező üres területeket feketével (átlátszóval) tölti ki. [16]

[17]

Alpha blending

A `blend()` függvény a kártya háttérképre helyezését végzi el. Az alábbi képlet segítségével:

$$I = \alpha F + (1 - \alpha)B$$

Alpha blending képlete [Alpha Blending Using OpenCV \(C++ / Python\) | LearnOpenCV #](#) 2026.03.11.

Ahol

- F az előtér színe
- B a háttér színe
- α az alfa maszk, ami az átlátszóságot reprezentálja és gyakran a kép negyedik csatornája. Az egyenletben használt érték az alfa maszkban lévő pixelérték osztva 255-tel. Tehát értéke az egyenletben $0 \leq \alpha \leq 1$. Amikor $\alpha=0$, a kimeneti pixel színe a háttér. Amikor $\alpha=1$, a kimeneti pixel színe az előtér. Amikor $0 < \alpha < 1$, a kimeneti pixel színe háttér és előtér keveréke. [18]


```

49  def blend(bg, card, x, y):
50      h, w = card.shape[:2]
51      roi = bg[y:y+h, x:x+w].astype(np.float32)
52      rgb = card[:, :, :3].astype(np.float32)
53      alpha = (card[:, :, 3].astype(np.float32) / 255.0)[:, :, None]
54      bg[y:y+h, x:x+w] = np.clip(alpha * rgb + (1 - alpha) * roi, 0, 255).astype(np.uint8)

```

generate_cards_dataset.py – blend függvény (saját kép) 2026.03.13.

A `def blend(bg, card, x, y)` függvénydefinícióban az `x, y` annak a pozíciónak a bal felső sarka, ahova a kártyát rá akarjuk helyezni.

A `h, w = card.shape[:2]` lekéri a kép magasságát és szélességét.

`roi = bg[y:y+h, x:x+w].astype(np.float32)` kivágja a háttérből azt a részt, ahova a kártya kerül, és átalakítja lebegőpontos formára, mert a blending képlete lebegőpontos műveleteket használ.

`rgb = card[:, :, :3].astype(np.float32)` kiveszi az kártyaképből (előtér) az első három csatornát (színét).

`alpha = (card[:, :, 3].astype(np.float32) / 255.0)[:, :, None]` kiválasztja a negyedik csatornát, vagyis az alpha csatornát, és osztja 255-tel, tehát a képlethez szükséges értékeket adja vissza.

`bg[y:y+h, x:x+w] = np.clip(alpha * rgb + (1 - alpha) * roi, 0, 255).astype(np.uint8)` a teljes művelet lényege. A fenti képlet alapján végrehajtja a kártya háttérképre helyezését.

Bounding box számítás

```

56  def alpha_bbox(card, x, y):
57      alpha = card[:, :, 3]
58      ys, xs = np.where(alpha > 10)
59      if len(xs) == 0:
60          return None
61      x1 = xs.min() + x
62      y1 = ys.min() + y
63      x2 = xs.max() + 1 + x
64      y2 = ys.max() + 1 + y
65      return x1, y1, x2, y2

```

generate_cards_dataset.py – alpha_bbox függvény (saját kép) 2026.03.13.

Ez a függvény kiszámolja a kártya bounding box-át csak az átlátszatlan pixelekből.

`alpha = card[:, :, 3]` kiveszi a negyedik csatornát, az alpha csatornát.

`ys, xs = np.where(alpha > 10)` megkeresi azokat a pixeleket amelyek nem átlátszók. `alpha>10` azért 10, mert előfordulhatnak zajos, 1-2 értékű pixelek.

`if len(xs) == 0: return None` ha a kép teljesen átlátszó, a függvény `None` értékkel tér vissza.

`A x1 = xs.min() + x, y1 = ys.min() + y, x2 = xs.max() + 1 + x és y2 = ys.max() + 1 + y` meghatározzák a legkisebb és legnagyobb x és y koordinátákat, és mivel a kártya már rá lett helyezve a háttérre, ezért a háttér pozíciót is hozzá kell adni eltolásként. Így például a `x1, y1` meg fogja adni a téglalap bal felső sarkát, míg a `x2, y2` a téglalap jobb alsó sarkát.

YOLO annotáció létrehozása

Ez a függvény a bounding box koordinátákat YOLO annotációs formátumba alakítja, és visszatér a szélességgel (`bw`), magassággal (`bh`) és középpont (`cx, cy`) koordinátákkal normalizált formában. Erre az átlalakításra azért van szükség, mert a YOLO modellek nem pixelkoordinátákkal dolgoznak, hanem normalizált középpont-koordinátákkal és méretekkel.

```
67  def yolo_label(class_id, box, img_size=640):
68      x1, y1, x2, y2 = box
69      bw = x2 - x1
70      bh = y2 - y1
71      cx = x1 + bw / 2
72      cy = y1 + bh / 2
73      return f"{class_id} {cx/img_size:.6f} {cy/img_size:.6f} {bw/img_size:.6f} {bh/img_size:.6f}"
```

generate_cards_dataset.py – yolo_label függvény (saját kép) 2026.03.13.

Mappaszerkezet létrehozása

Előre létrehozza a YOLO dataset könyvtárstruktúráját, hogy a képek és a label fájlok mentése során biztosan létezzenek a szükséges mappák.

```
75  def make_dirs():
76      for split in ["train", "val", "test"]:
77          (DATASET_DIR / "images" / split).mkdir(parents=True, exist_ok=True)
78          (DATASET_DIR / "labels" / split).mkdir(parents=True, exist_ok=True)
```

generate_cards_dataset.py – make_dirs függvény (saját kép) 2026.03.13.

Egyetlen generált kép létrehozása

`def generate_one_image()` függvény a dataset generátor központi eleme, mert ez készít el egy teljes szintetikus képet a hozzá tartozó YOLO címkékkel.

```

80  def generate_one_image():
81      bg = random.choice(backgrounds).copy()
82      chosen_cards = random.sample(class_names, 5)
83
84      placed_centers = []
85      labels = []
86
87      for name in chosen_cards:
88          card = cards[name]
89
90          scale = random.uniform(0.30, 0.45)
91          h, w = card.shape[:2]
92          new_h = int(scale * IMG_SIZE)
93          sc = new_h / h
94          card = cv2.resize(card, (int(w * sc), new_h))
95
96          card = rotate(card, random.uniform(-35, 35))
97          ch, cw = card.shape[:2]
98
99          for _ in range(50):
100             x = random.randint(0, IMG_SIZE - cw)
101             y = random.randint(0, IMG_SIZE - ch)
102             cx, cy = x + cw / 2, y + ch / 2
103
104             if any((cx - px) ** 2 + (cy - py) ** 2 < 40 ** 2 for px, py in placed_centers):
105                 continue
106
107             placed_centers.append((cx, cy))
108             blend(bg, card, x, y)
109
110             box = alpha_bbox(card, x, y)
111             if box is not None:
112                 labels.append(yolo_label(class_to_id[name], box))
113             break
114
115      return bg, labels

```

generate_cards_dataset.py – generate_one_image függvény (saját kép) 2026.03.13.

A függvény nem kap paramétert, mert a szükséges adatokat már a külső változókból használja: `backgrounds`, `cards`, `class_names`, `IMG_SIZE`, `class_to_id`.

`bg = random.choice(backgrounds).copy()` kiválaszt egy darab véletlen háttérképet.

`chosen_cards = random.sample(class_names, 5)` kiválaszt 5 darab különböző véletlen kártyát.

`placed_centers = []` egy üres lista amibe a már elhelyezett kártyák középpontjait kerülnek. Ezt később arra használja, hogy az új kártyák ne kerüljenek túl közel a korábbiakhoz.

`labels = []` ebbe a listába kerülnek a generált képhez tartozó YOLO annotációs sorok. Minden sikeresen elhelyezett kártyához kerül bele egy szövegsor.

A `for name in chosen_cards` ciklus végigmegy az előzőleg kiválasztott 5 kártyánéven, minden lapra külön-külön **méretezést** (az eredeti méret 30-45%-a között `scale = random.uniform(0.30, 0.45)`), **forogatást** (-35 és 35 fok között `card = rotate(card, random.uniform(-35, 35))`) és **pozíciót** választ.

A pozícióválasztás során kiszámolja az aktuális még csak tervezett kártya középpontot `cx, cy = x + cw / 2, y + ch / 2`.

`if any((cx - px) ** 2 + (cy - py) ** 2 < 40 ** 2 for px, py in placed_centers)` ellenőrzi, hogy az aktuálisan kiválasztott középpont 40 képpontnál közelebb van-e bármelyik előző kártya középpontjához. Ha igen, akkor ezt a pozíciót eldobja, és jön az 50 próbálkozásból a következő. Ha megfelelő, akkor elmentjük `placed_centers.append((cx, cy))`, ráhelyezzük a háttérre `blend(bg, card, x, y)`, kiszámítja a kártya tényleges bounding boxát `box = alpha_bbox(card, x, y)`, majd YOLO annotációs sort készít `labels.append(yolo_label(class_to_id[name], box))`. Végül visszaadja a már kártyákkal feltöltött képet és a hozzá tartozó YOLO annotációs sorok listáját `return bg, labels`.

Dataset mentése

A `def save_split(split, count, start_idx=0)` függvény a dataset generátor utolsó lépése. Az előbb elkészített képeket és a hozzájuk tartozó YOLO annotációkat fájlokba menti, mégpedig a megfelelő dataset részbe (`train`, `val`, `test`) amelyet a `split` paraméter ad meg. A `count` paraméter a generált képek számát, a `start_idx` pedig a fájlnevek kezdő indexét adja meg.

```

117  def save_split(split, count, start_idx=0):
118      for i in range(count):
119          img, labels = generate_one_image()
120          idx = start_idx + i
121
122          img_path = DATASET_DIR / "images" / split / f"img_{idx:05d}.jpg"
123          txt_path = DATASET_DIR / "labels" / split / f"img_{idx:05d}.txt"
124
125          cv2.imwrite(str(img_path), img)
126
127          with open(txt_path, "w") as f:
128              for line in labels:
129                  f.write(line + "\n")

```

generate_cards_dataset.py – save_split függvény (saját kép) 2026.03.13.

Végül a program létrehozza a dataset mappastruktúráját, és generál 4000 tanító (**train**), 500 validációs (**val**) és 500 teszt (**test**) képet.

```

131  make_dirs()
132
133  save_split("train", 4000, 0)
134  save_split("val", 500, 4000)
135  save_split("test", 500, 4500)

```

generate_cards_dataset.py részlet (saját kép) 2026.03.13.

Training.py program bemutatása

Ez a kód a modell tanítását Google Colab-ban és a tanított modell letöltését végzi.

```

1  from ultralytics import YOLO
2
3  model = YOLO("yolov8n.pt")
4  model.train(
5      data="cards.yaml",
6      imgsz=640,
7      epochs=50,
8      batch=-1,
9      device=0,
10     project="runs",
11     name="card_detector"
12 )
13 from google.colab import files
14 files.download('/runs/card_detector/weights/best.pt')

```

training.py (saját kép) 2026.03.13.

A `from ultralytics import YOLO` betölti az Ultralytics könyvtár YOLO osztályát. A YOLO osztály egy magas szintű API ami lehetővé teszi a modell betöltését, tanítását, tesztelését és predikció futtatását.

A `model = YOLO("yolov8n.pt")` létrehoz egy YOLO modell objektumot, ahol a `YOLO(...)` az osztály konstruktora, amelynek paramétere (`"yolov8n.pt"`) egy előtanított modell fájl amely a nano vagyis a legkisebb modell. Ennek előnyei a gyors tanítás és kevés memória használat.

A `model.train()` meghívja a YOLO API `train()` függvényét ami elindítja a training pipeline-t.

A `data="cards.yaml"` megmondja a YOLO-nak hol van a dataset konfigurációja.

```
1 path: datasets/yolo_cards
2 train: images/train
3 val: images/val
4 test: images/test
5
6 names:
7   0: C10
8   1: C2
9   2: C3
10  3: C4
```

cards.yaml részlet (saját kép) 2026.03.13.

Az `imgsz=640` a tanítás során használt input képméret.

Az epoch a dataset egy teljes feldolgozását jelenti, az `epochs=50` 50 feldolgozást jelent.

A `batch=-1` automatikus batch méretet jelent, vagyis a program megpróbálja a GPU memóriát teljesen kihasználni.

A `device=0` megmondja, hogy GPU-t használjunk.

A `project="runs"` tartalmazza a training eredményeket.

A `name="card_detector"` azt jelenti, hogy a `runs/card_detector/` mappában lesznek a legjobb validációs eredmény (`best.pt`) és az utolsó epoch eredménye (`last.pt`).

Végül letölti a validációs adatokon legjobban teljesítő tanított modellt a gépünkre `files.download('/runs/card_detector/weights/best.pt')`

[13][19]

Eredmények

A modell tanítása három különböző architektúrával is megtörtént: **Ultralytics YOLOv8n**, **YOLO11n** és **YOLO12n**. A tanítás során a korábban generált szintetikus adathalmazt használtuk, amely 640x640 pixeles képeket tartalmaz. A dataset három részre lett bontva, tanító (train), validációs (val) és teszt (test) halmazokra. A modellek tanítása 50 epochon keresztül zajlott.

A modellek betanítása során a rendszer folyamatosan számolta a veszteségfüggvények értékeit, valamint a validációs metrikákat. A tanítás végére a veszteségértékek stabilan csökkentek, ami azt jelzi, hogy a modellek sikeresen megtanulták a kártyák detektálását és osztályozását.

```
50 epochs completed in 0.986 hours.
Optimizer stripped from /content/runs/detect/runs/card_detector11/weights/last.pt, 6.3MB
Optimizer stripped from /content/runs/detect/runs/card_detector11/weights/best.pt, 6.3MB

Validating /content/runs/detect/runs/card_detector11/weights/best.pt...
Ultralytics 8.4.21 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Model summary (fused): 73 layers, 3,015,788 parameters, 0 gradients, 8.1 GFLOPs
Class      Images  Instances  Box(P    R    mAP50  mAP50-95): 100% 10/10 1.4it/s 7.1s
  all         600      3000      0.98    0.923  0.975   0.952
```

Training eredmények YOLOv8n (saját kép) 2026.03.13.

```
50 epochs completed in 1.051 hours.
Optimizer stripped from /content/runs/detect/runs/card_detector/weights/last.pt, 5.5MB
Optimizer stripped from /content/runs/detect/runs/card_detector/weights/best.pt, 5.5MB

Validating /content/runs/detect/runs/card_detector/weights/best.pt...
Ultralytics 8.4.25 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLO11n summary (fused): 101 layers, 2,592,292 parameters, 0 gradients, 6.4 GFLOPs
Class      Images  Instances  Box(P    R    mAP50  mAP50-95): 100% 12/12 1.4it/s 8.3s
  all         600      3000      0.985    0.937  0.979   0.964
```

Training eredmények YOLO11n (saját kép) 2026.03.23.

```
50 epochs completed in 1.200 hours.
Optimizer stripped from /content/runs/detect/runs/card_detector/weights/last.pt, 5.5MB
Optimizer stripped from /content/runs/detect/runs/card_detector/weights/best.pt, 5.5MB

Validating /content/runs/detect/runs/card_detector/weights/best.pt...
Ultralytics 8.4.26 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLOv12n summary (fused): 159 layers, 2,566,868 parameters, 0 gradients, 6.4 GFLOPs
Class      Images  Instances  Box(P    R    mAP50  mAP50-95): 100% 17/17 2.1it/s 8.0s
  all         600      3000      0.99    0.934  0.981   0.965
```

Training eredmények YOLO12n (saját képe) 2026.03.24.

Modellek komplexitási jellemzői

Modell	Paraméterszám	GFLOPs	Modellméret (MB)
YOLOv8n	3,015,788	8.1	6.3
YOLO11n	2,592,292	6.4	5.5
YOLO12n	2,566,868	6.4	5.5

A táblázatban szereplő mutatók jelentése:

- **Paraméterszám:** a modellben lévő „súlyok” (kapcsolatok) száma

- **GFLOPs:** a modell számítási igénye, amely azt mutatja, hogy egy kép feldolgozása során hány milliárd lebegőpontos műveletet hajt végre.
- **Modellméret:** a tanított modell fájlmérete, amely befolyásolja a tárolási igényt és a betöltési sebességet.

A fenti mutatók alapján megállapítható, hogy a modellek a könnyűsúlyú kategóriába tartoznak, azaz alacsony számítási és tárolási igénnyel rendelkeznek. A további mutatók alapján pedig azt is láthatjuk, hogy emellett képesek magas pontosságot elérni. [24]

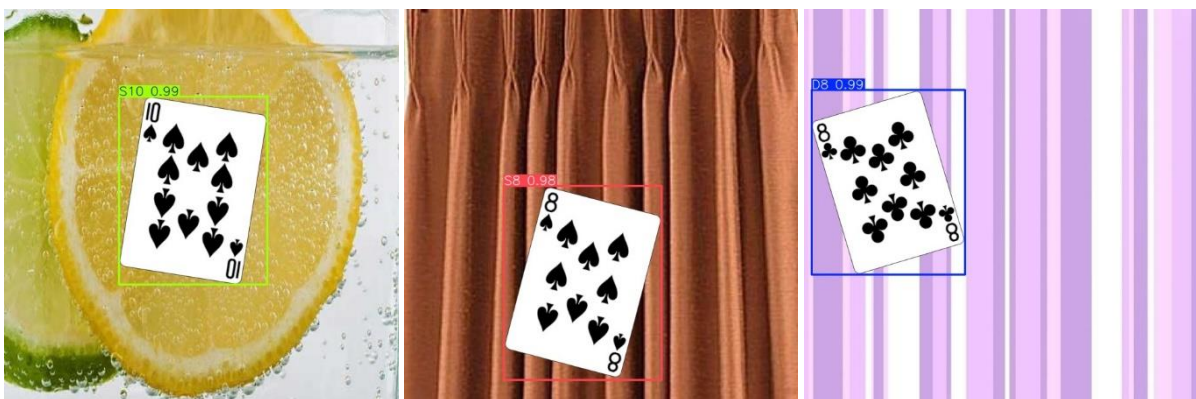
Modellek teljesítménye

A modellek teljesítményének vizsgálatához három különböző nehézségű adathalmazt definiáltam. A **Test easy** képenként 1, a **Test medium** képenként 3, a **Test hard** pedig képenként 5 különböző kártyát tartalmazott. Ezzel azt vizsgáltam, hogy a modellek hogyan teljesítenek növekvő komplexitás mellett.

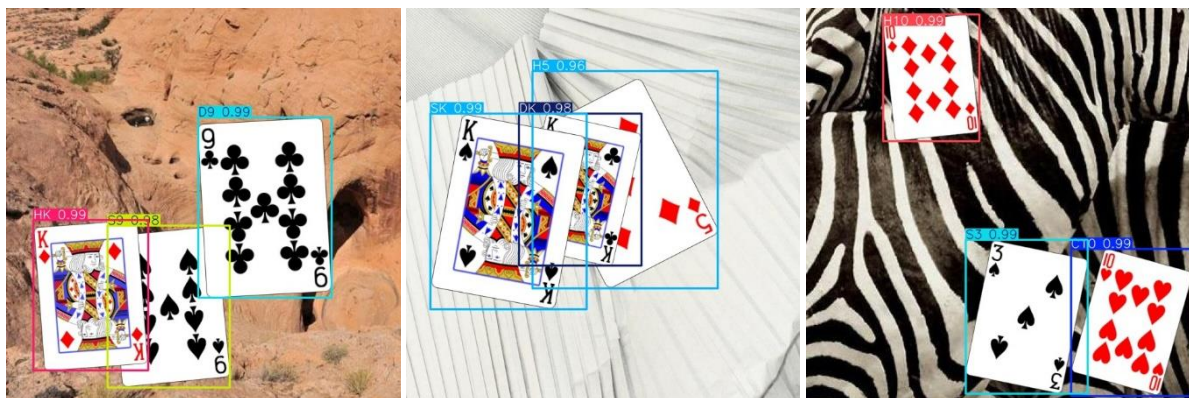
A validációs adathalmazokon kapott összesített eredmények az alábbiak voltak:

YOLOv8n	Test easy	Test medium	Test hard
Precision	0.993	0.991	0.98
Recall	1	0.985	0.923
mAP50	0.995	0.994	0.975
mAP50-95	0.995	0.988	0.952

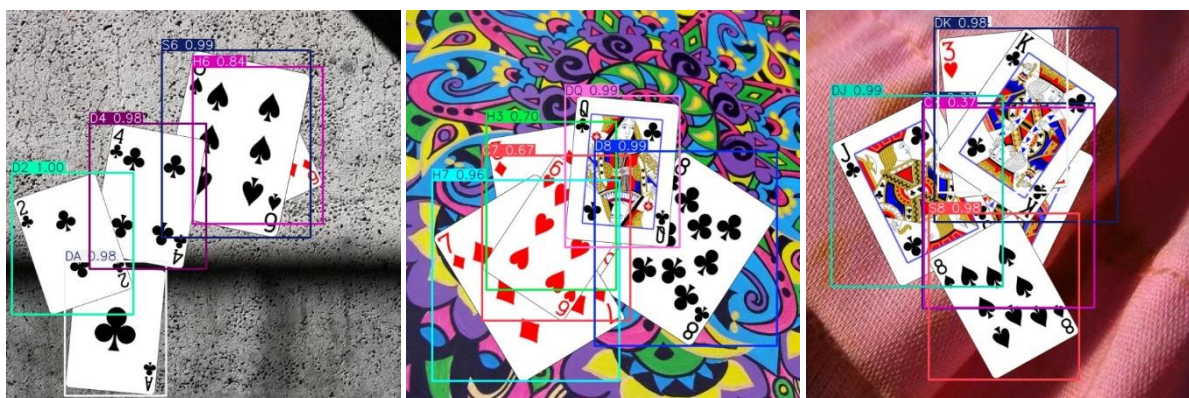
Az alábbiakban néhány detekció látható a YOLOv8n modellel a Test easy, Test medium, Test hard adathalmazokon.



YOLOv8n detekciók a Test easy adathalmazon 2026. 05. 11.



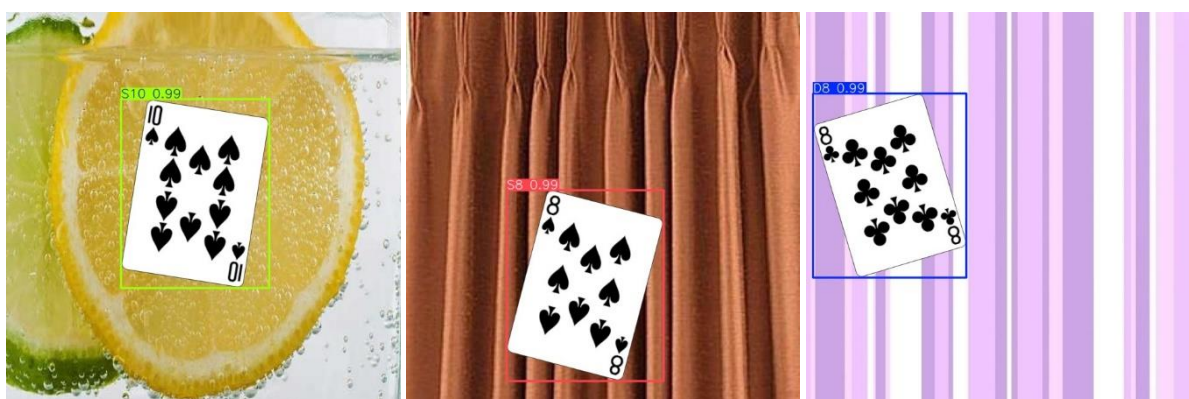
YOLOv8n detekciók a Test medium adathalmazon 2026. 05. 11.



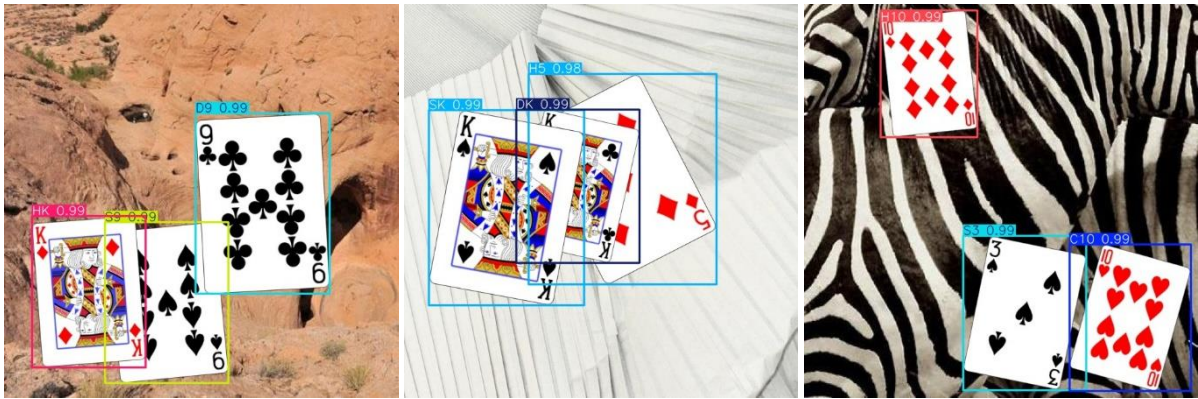
YOLOv8n detekciók a Test hard adathalmazon 2026. 05. 11.

YOLOv11n	Test easy	Test medium	Test hard
Precision	0.994	0.992	0.985
Recall	1	0.989	0.937
mAP50	0.995	0.995	0.979
mAP50-95	0.995	0.992	0.964

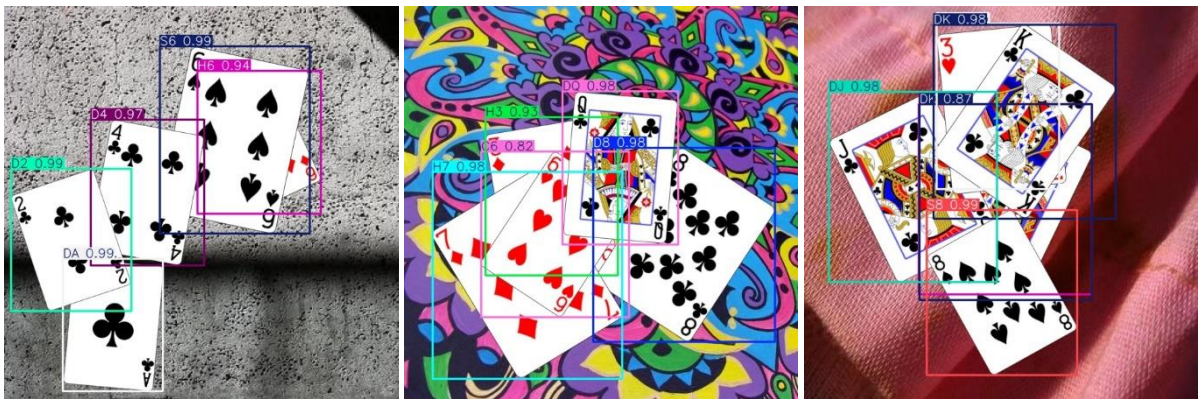
Az alábbiakban néhány detekció látható a YOLOv8n modellel a Test easy, Test medium, Test hard adathalmazokon.



YOLOv11n detekciók a Test easy adathalmazon 2026. 05. 11.



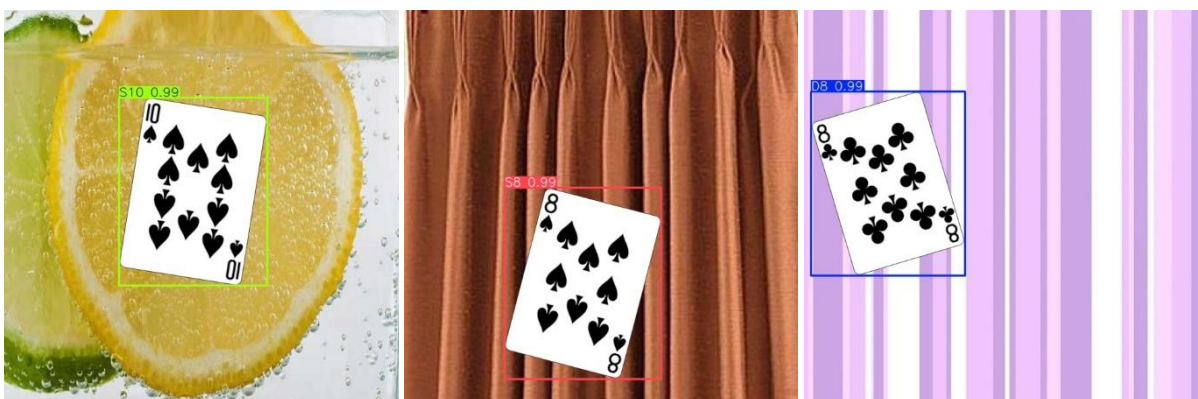
YOLOv11n detekciók a Test medium adathalmazon 2026. 05. 11.



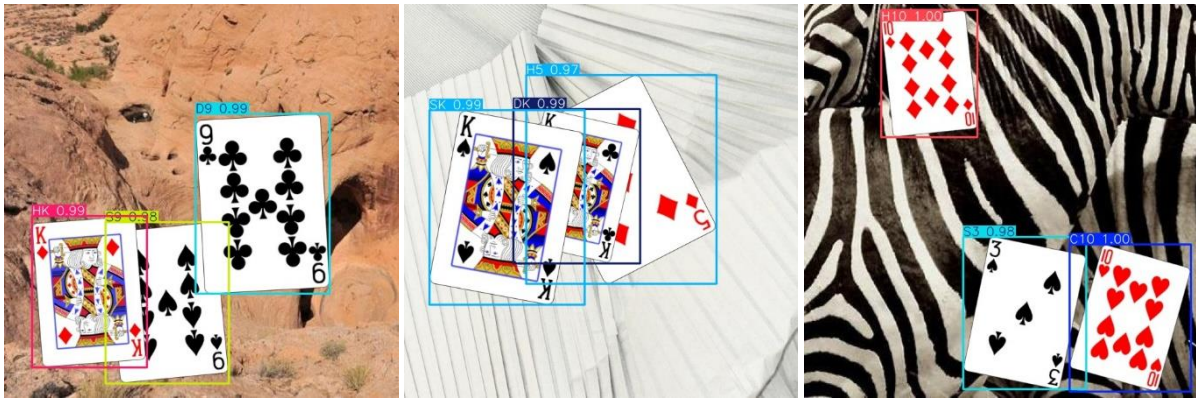
YOLOv11n detekciók a Test hard adathalmazon 2026. 05. 11.

YOLOv12n	Test easy	Test medium	Test hard
Precision	0.993	0.991	0.99
Recall	1	0.991	0.934
mAP50	0.995	0.994	0.981
mAP50-95	0.995	0.99	0.965

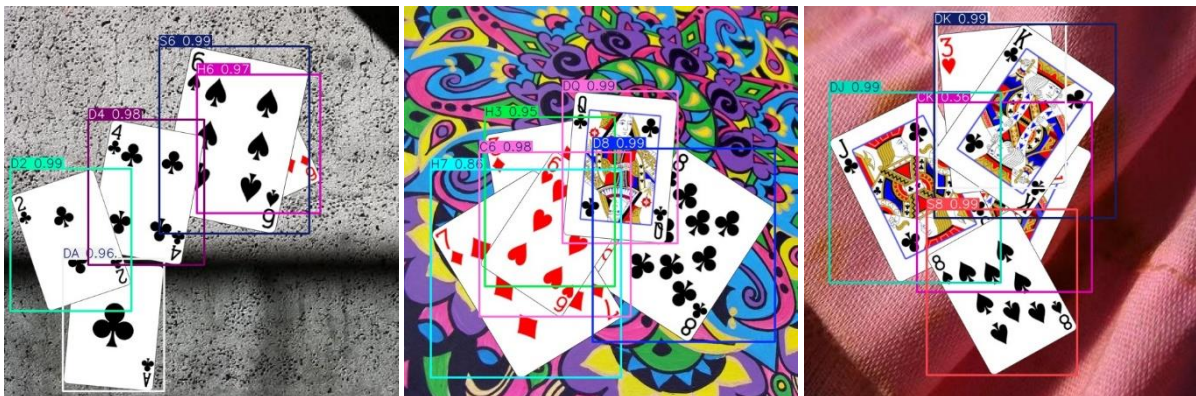
Az alábbiakban néhány detekció látható a YOLOv8n modellel a Test easy, Test medium, Test hard adathalmazokon.



YOLOv12n detekciók a Test easy adathalmazon 2026. 05. 11.



YOLOv12n detekciók a Test medium adathalmazon 2026. 05. 11.



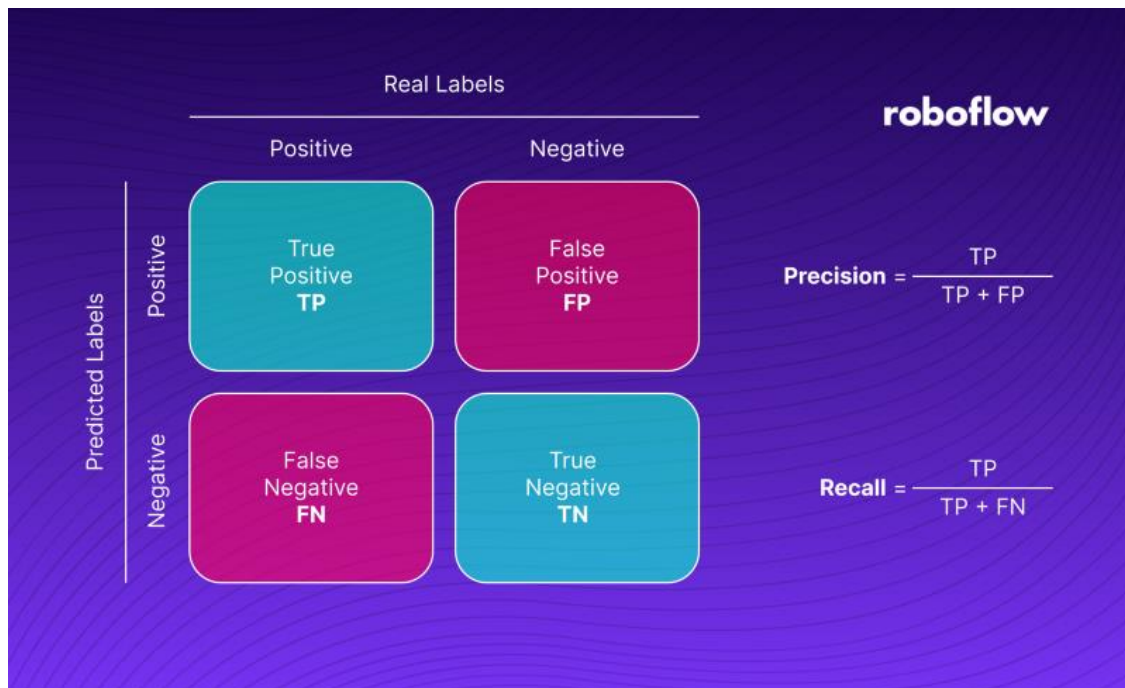
YOLOv12n detekciók a Test hard adathalmazon 2026. 05. 11.

Az eredmények alapján megfigyelhető, hogy mindegyik modell nagyon magas pontosságot ért el. A nehezebb esetekben a teljesítmény enyhén csökken, különösen a recall értékeknél, ami arra utal, hogy több objektum és átfedések esetén nő a kihagyott detekciók száma.

A fejlettebb modellek általában enyhén jobb eredményt mutatnak, de megfigyelhető, hogy a YOLO11n modell egyes esetekben kissé jobb eredményt ért el, mint a YOLO12n. Ez ellentmondásosnak tűnhet, mert az újabb architektúrák általában jobb teljesítményt nyújtanak. Ennek egyik oka lehet, hogy a modellek tanítása során több véletlenszerű tényező is szerepet játszik. Megbízhatóbb összehasonlítás érdekében több tanítás átlagára lenne szükség.

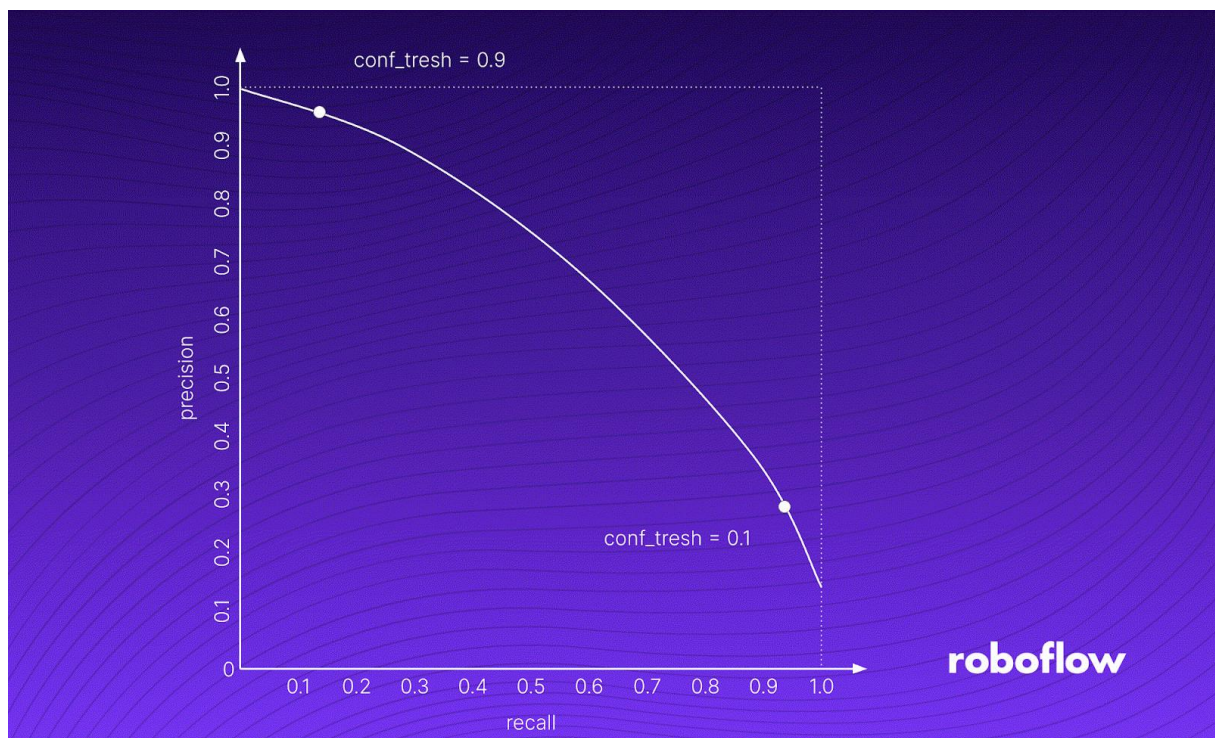
Metrikák magyarázata

A **precision** azt mutatja meg, hogy a modell által detektált objektumok mekkora része valóban helyes detekció. A **recall** azt méri, hogy a valós objektumok mekkora részét találta meg a modell.



Precision és Recall [mAP in Object Detection: Mean Average Precision Explained](#) 2026.03.13.

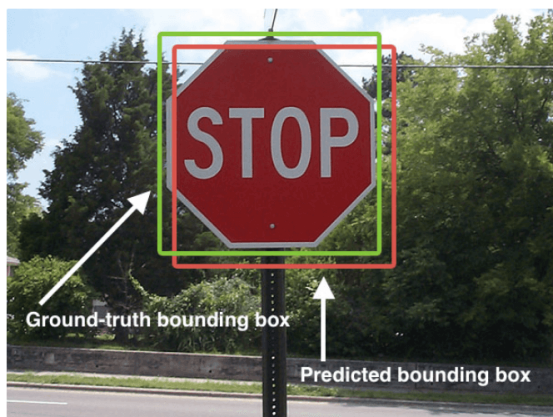
A **mean Average Precision (mAP)** a modern objektumdetektáló rendszerek egyik legfontosabb teljesítménymérő mutatója, amely a detekció pontosságát és a bounding box lokalizáció pontosságát egyaránt figyelembe veszi. Az **AP** a Precision átlaga az összes lehetséges Recall szinten. A **mAP** pedig az összes szint AP értékének az **átlaga**.



Average precision az összes recall szinten [mAP in Object Detection: Mean Average Precision Explained](#) 2026.03.13.

A **mAP50** esetében minden olyan találatot jónak fogadunk el, ahol az **IoU legalább 0,5 (50%)**. Azt mutatja, mennyire jó a modell az objektumok "megtalálásában", kevésbé szigorú a pontos elhelyezkedéssel szemben.

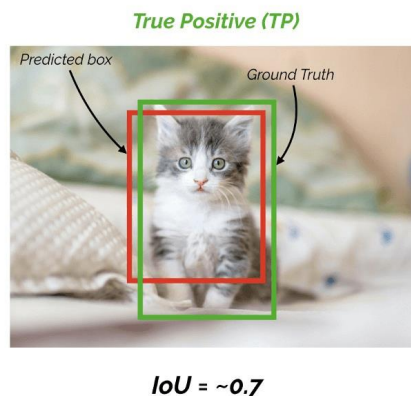
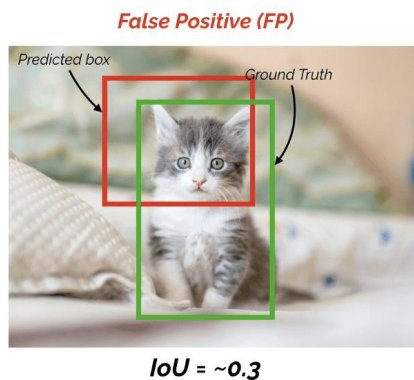
Az **IoU** azt méri, mennyire fedi át egymást a modell által rajzolt keret és a valós keret. Úgy számoljuk, hogy a két téglalap közös területét elosztjuk az általuk lefedett összesített területtel. Ha az érték 1.0, a két keret tökéletesen fedi egymást.



$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

Intersection over Union (IoU) [Intersection over Union \(IoU\)](#) 2026.03.13.

If IoU threshold = 0.5



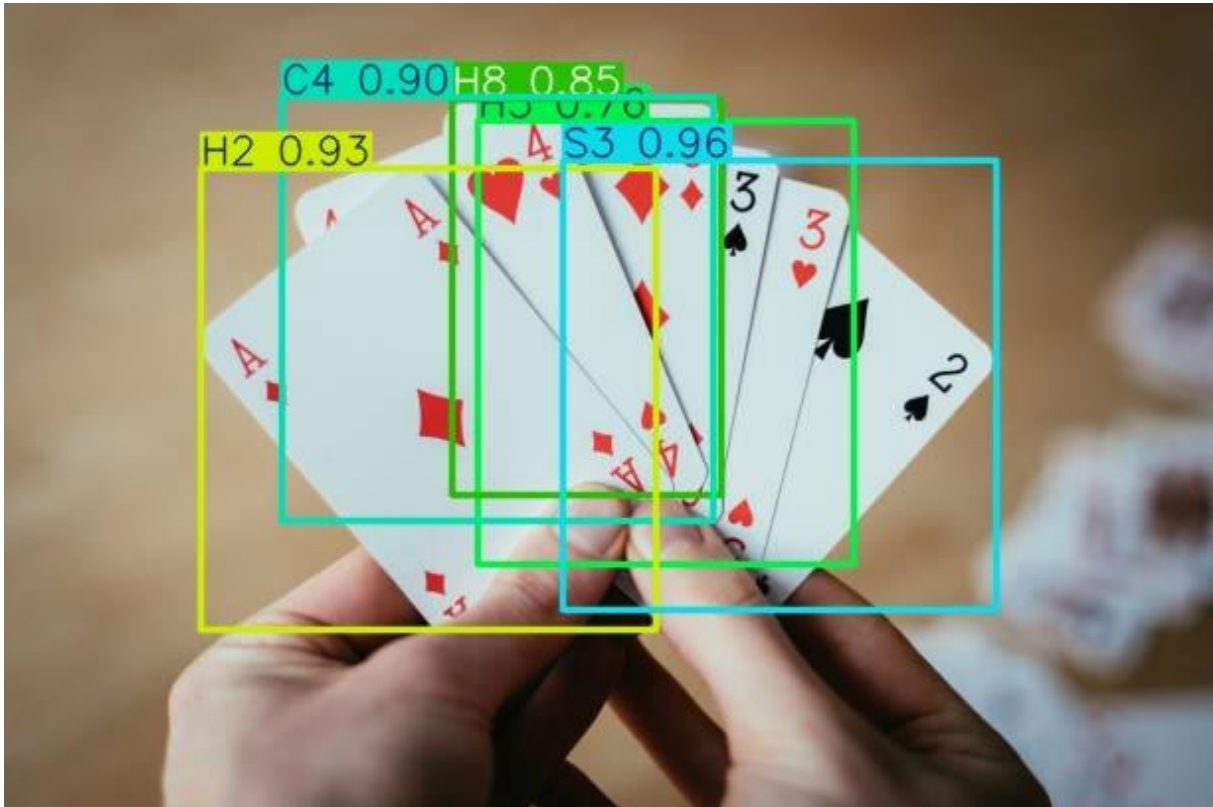
If IoU threshold = 0.5 [Mean Average Precision \(mAP\) Explained: Everything You Need to Know](#) 2026.03.13.

A **mAP50-95** a mAP átlaga több IoU küszöbön (0,5-től 0,95-ig, 0,05-ös lépésekkel).

Az elért **mAP@0.5 = 0.975** feletti értékek azt jelzik, hogy a modell a validációs adatokon nagyon magas pontossággal képes felismerni a kártyákat. A szigorúbb **mAP@0.5-0.95 = 0.952** feletti értékek szintén kiemelkedően jó teljesítményt mutatnak.

A legtöbb kártyaosztály esetében a precision és recall értékek is 0.9 fölött alakultak, ami azt mutatja, hogy a modell megbízhatóan képes felismerni a különböző kártyalapokat. Fontos azonban megjegyezni, hogy a tanítás során használt adatok **szintetikus**

generált képekből származnak, ezért a modell teljesítménye valós fotókon eltérhet. Valós környezetben a háttér, a megvilágítás, valamint a részleges takarások változatosabbak lehetnek. Emiatt a modell további javítása érdekében érdemes lenne a datasetet valós képekkel bővíteni és a modellt újra finomhangolni.



Valós képen detektálási hiányosságok és alacsony confidence score-ok a YOLOv12n modellel (saját kép) 2026.03.13.

[20] [21] [22]

A modell használata predikcióra

A betanított YOLO modell a tanítás után közvetlenül használható új képek elemzésére. A predikció során a modell egy bemeneti képen automatikusan detektálja az objektumokat, meghatározza azok pozícióját (bounding box), valamint az objektum osztályát. Ehhez az alábbi kódban a `model = YOLO(...)`-ba be kell illeszteni a `best.pt` elérési útját, a `source` paraméter helyére pedig a feldolgozandó kép elérési útját.

```
1 from ultralytics import YOLO
2
3 model = YOLO("C:/Downloads/best.pt")
4 model.predict(source="C:/card-detector/datasets/yolo_cards/images/test/img_00922.jpg", save=True)
```

prediction.py (saját kép) 2026.03.13.

[23]

Hivatkozások

- [1] Simon Haykin, Neural Network and Learning Machines Third Edition, Models of a Neuron [2026.03.07] [DeepLearning/Neural Networks and Learning Machines \(3rd Edition\).pdf at master · xinlin192/DeepLearning](#)
- [2] IBM What is deep learning?, How deep learning works [2026.03.07] [What Is Deep Learning? | IBM](#)
- [3] Mohammad Mustafa Taye, Theoretical Understanding of Convolutional Neural Network: Concepts, Architectures, Applications, Future Directions 2.1 CNN fundamentals 2026.03.07
- [4] Aston Zhang, Zachary C. Lipton, Mu Li, Alexander J. Smola, Dive into deep learning, 7. Convolutional Neural Networks [7.1. From Fully Connected Layers to Convolutions — Dive into Deep Learning 1.0.3 documentation](#)
- [5] Aston Zhang, Zachary C. Lipton, Mu Li, Alexander J. Smola, Dive into deep learning, 7. Convolutional Neural Network [7.2. Convolutions for Images — Dive into Deep Learning 1.0.3 documentation](#)
- [6] Benyó Balázs, Konvolúciós hálózatok előadás [2026.03.07] [Convolut_network_Benyo.pdf](#)
- [7] Aston Zhang, Zachary C. Lipton, Mu Li, Alexander J. Smola, Dive into deep learning, 7. Convolutional Neural Networks [7.5. Pooling — Dive into Deep Learning 1.0.3 documentation](#)
- [8] Medium, Muhammad Shoaib Ali [2026.03.07] [Flattening CNN layers for Neural Network and basic concepts | by Muhammad Shoaib Ali | Medium](#)
- [9] Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi, You Only Look Once: Unified, Real-Time Object Detection, 1.Introduction [2026.03.07] [You Only Look Once: Unified, Real-Time Object Detection](#)
- [10] Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi, You Only Look Once: Unified, Real-Time Object Detection 2. Unified Detection [2026.03.07] [You Only Look Once: Unified, Real-Time Object Detection](#)
- [11] Ultralytics, Object Detection Datasets Overview, Ultralytics YOLO format [2026.03.07] [Object Detection Datasets Overview - Ultralytics YOLO Docs](#)
- [12] Manuel Carranza-García, Jesús Torres-Mateo, Pedro Lara-Benítez, Jorge García-Gutiérrez, On the Performance of One-Stage and Two-Stage Object Detectors in Autonomous Vehicles Using Camera Data [2026.03.07] [On the Performance of One-Stage and Two-Stage Object Detectors in Autonomous Vehicles Using Camera Data](#)

- [13] Ultralytics, Explore Ultralytics YOLOv8 [2026.03.13] [Explore Ultralytics YOLOv8 - Ultralytics YOLO Docs](#)
- [14] Georgios Georgakis, Arsalan Mousavian, Alexander C. Berg, Jana Kosecka, Synthesizing Training Data for Object Detection in Indoor Scenes [1702.07836](#)
- [15] OpenCV: getRotationMatrix2D() [2026.03.08] [OpenCV: Geometric Image Transformations](#)
- [16] OpenCV: warpAffine() [2026.03.08] [OpenCV: Geometric Image Transformations](#)
- [17] OpenCV: Image Rotation and Translation using OpenCV, Example of avoiding cutoff by enlarging canvas size [2026.03.08] [Image Rotation and Translation using OpenCV](#)
- [18] LearnOpenCV Sunita Nayak, Alpha Blending Using OpenCV (C++ / Python) [2026.03.11] [Alpha Blending Using OpenCV \(C++ / Python\) | LearnOpenCV #](#)
- [19] Ultralytics, Model Training with Ultralytics YOLO [2026.03.13] [Model Training with Ultralytics YOLO - Ultralytics YOLO Docs](#)
- [20] Ultralytics, Performance Metrics Deep Dive [2026.03.13] [Performance Metrics Deep Dive - Ultralytics YOLO Docs](#)
- [21] Roboflow, Jacob Solawetz, What is Mean Average Precision (mAP) in Object Detection? [2026.03.13] [mAP in Object Detection: Mean Average Precision Explained](#)
- [22] V7labs, Deval Shah, Mean Average Precision (mAP) Explained: Everything You Need to Know [2026.03.13] <https://www.v7labs.com/blog/mean-average-precision>
- [23] Ultralytics, Model Prediction with Ultralytics YOLO [2026.03.13] [Model Prediction with Ultralytics YOLO - Ultralytics YOLO Docs](#)
- [24] Ultralytics, FLOPs [2026.03.24] [What are FLOPs? Model Complexity & Metrics | Ultralytics](#)